

Lecture Note: OOPs Concepts – Basics



Nishut Suman

17 Dec, 2025 At 5:00 PM

Notes

Foundation

Recommended

Resource



Description



Discussions

Lecture Notes: Introduction to Object-Oriented Programming (OOPs Basics)

Prerequisites:

Understanding of Python variables, data types (strings, integers, lists), and defining basic functions.

What you'll be able to do:

- Explain the core principles and need for Object-Oriented Programming.
- Define and implement Python classes, objects, constructors, attributes, and methods.
- Compare procedural programming with object-oriented programming.
- Write small OOP-based Python programs that model real-world entities.

1. Introduction: What is Object-Oriented Programming (OOP) and Why Should You Care?

Core Definition

Object-Oriented Programming (OOP) is a programming paradigm where data and the operations that manipulate that data are bundled together into units called **objects**.

Each object is created from a **class**, which acts as a **blueprint** defining the structure (attributes) and behavior (methods).

In Python, OOP makes code more **modular, reusable, and reflective** of real-world entities.

A Simple Analogy

Think of a **class** as a car design blueprint - it defines what a car *is* (like wheels, engine, color).

An **object** is an *actual car* built from that blueprint, such as your personal red Tesla.

 **Limitation:** The analogy doesn't fully capture dynamic behaviors like inheritance or polymorphism, which go beyond static **blueprints**.

Why This Matters to You

Problem it solves: Procedural code becomes messy when applications grow large - OOP helps organize and structure it logically.

What you'll gain:

- Write scalable programs by grouping data and behavior together.
- Reuse code easily using classes and objects.
- Model real-world problems naturally (e.g., a BankAccount or Student).

Real-world context: Frameworks like Django, Flask, and even data science libraries like Pandas use OOP principles internally.

2. The Foundation: Core Concepts Explained

Concept A: Class

Definition: A **class** in Python defines the blueprint or template for creating objects. It contains attributes (data) and methods (functions).

Key characteristics:

- Defines *what* an object will have (attributes).
- Defines *what* an object can do (methods).
- Created using the **class** keyword.

Example:

```
class Dog:
    pass
```

pass keyword: We use pass when we don't want to write any code right now but want the program to run without error. It acts like a placeholder for future code. Here, we wrote pass because we will add code later, but for now, it lets the program run without any error.

Common confusion: Beginners often think a class itself *is* an object - it's not. The class is the **plan**, and the object is the **product**.

Concept B: Object

Definition: An **object** is an instance of a class-an actual realization of the class blueprint. Each object has its own data.

How it relates to Class: A class defines the structure; an object brings it to life.

Key characteristics:

- Has attributes and methods defined by its class.
- Each object can store unique data.
- Created by calling the class name like a function.

Example:

```
my_dog = Dog()
```

Remember: Objects are independent-creating multiple objects doesn't change the class definition.

Concept C: Constructor (*init*)

(Here use 2-2 underscores __ both the sides)

Definition: A constructor is a special method named `__init__` that initializes object attributes when it is created.

Key characteristics:

- Runs automatically when you create an object.
- Defines the object's initial state.
- Uses **self** to refer to the current object.

Example:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

Common confusion: self is not a keyword, it's just a naming convention. It refers to *this particular instance* of the class.

Concept D: Attributes and Methods**Attributes (Data):**

- Variables that hold the object's state or data.
- Defined in the constructor or directly inside the class.

Methods (Behaviors):

- Functions inside a class that define actions an object can perform.

Example:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")
```

How Class, Object, Constructor, and Methods work together:

Think of a class as a **blueprint**, a constructor as the **building process**, and attributes/methods as the **features and actions** of the finished object.

3. Seeing It in Action: Worked Examples

Example 1: The Basic Case

Scenario: Creating a Dog class and making it bark.

Step-by-step solution:

```
# Step 1: Define the class
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")

# Step 2: Create an object (instance)
dog1 = Dog("Buddy", "Labrador")

# Step 3: Call the method
dog1.bark()
```

Output:

Buddy says Woof!

What just happened: We defined a Dog blueprint, created a Dog object named **dog1**, and called its **bark()** method. The constructor initialized its name and breed.

Example 2: Adding Complexity

Scenario: Adding an age attribute and method to display it.

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name
        self.breed = breed
        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

    def display_info(self):
        print(f"{self.name} is a {self.age}-year-old {self.breed}.")

dog2 = Dog("Charlie", "Beagle", 4)
dog2.display_info()
```

Output:

```
Charlie is a 4-year-old Beagle.
```

Key lesson: Adding new attributes and methods expands functionality - this is the power of OOP scalability.

Example 3: Real-World Application

Background: Modeling a student management system.

Challenge: Represent each student with personal info and GPA.

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa

    def introduce(self):
        print(f"Hi, I'm {self.name} and my GPA is {self.gpa}.")

student1 = Student("Arjun", 8.9)
student2 = Student("Meera", 9.3)

student1.introduce()
```

```
student2.introduce()
```

Output:

```
Hi, I'm Arjun and my GPA is 8.9.  
Hi, I'm Meera and my GPA is 9.3.
```

Caution: If you forget **self**, Python won't know which object you're referring to - always include it in method definitions.

4. Common Pitfalls: What Can Go Wrong and How to Avoid It

- **The Mistake:** Forgetting **self** in method definitions
 - **Why It's a Problem:** Python raises `TypeError: missing 1 required positional argument`
 - **The Right Approach:** Always include **self** as the first parameter in methods.
 - **Why This Works:** **self** ensures methods access attributes of the current object.
- **The Mistake:** Confusing class variables with instance variables
 - **Why It's a Problem:** Changes to one object affect all others unexpectedly.
 - **The Right Approach:** Use `self.attribute` for instance variables.
 - **Why This Works:** Each object gets its own copy of data.
- **The Mistake:** Thinking constructors return objects
 - **Why It's a Problem:**
`*init*` doesn't return anything - it only initializes.
 - **The Right Approach:** Don't use **return** inside `*init*`
 - **Why This Works:** Python handles object creation internally; `*init*` just prepares it.

If you're stuck:

Revisit the **Class** and **Constructor** sections, and practice small examples to visualize how instances hold their own data.

5. Your Turn: Practice & Self-Assessment

Practice Task

The Challenge: Create a Python class **Book** that represents a library book.

Specifications:

- The constructor should take **title, author, and price**.
- Add a method **display_info()** that prints: "Title: [title], Author: [author], Price: ₹[price]"
- Instantiate two books and call the method on both.

Hint: Think about how each book object will have its own independent data stored as instance variables.

Extension: Add a method **discounted_price(percent)** that prints the price after a given discount.

Check Your Understanding

Explanation: Why do we use **self** in Python class methods?

Application: What happens if you define a method without including **self**?

Error analysis: What is wrong with this code?

```
class Dog:
    def __init__(name):
        self.name = name
```

Transfer: How would you model a **BankAccount** class using what you learned?

Answers & Explanations:

self represents the current instance - it lets each object maintain its own data.

Python throws an error because the method won't receive the instance automatically.

The constructor is missing the **self** parameter.

Create attributes like **balance** and methods like **deposit()** or **withdraw()** using the OOP structure.

6. Consolidation: Key Takeaways & Next Steps

The Essential Ideas

- **Principle 1:** A *class* is a blueprint; an *object* is an instance of that blueprint.
- **Key Application:** Constructors initialize attributes that define object state.
- **Critical Warning:** Always use **self** to reference instance variables.

Mental Model Check

Think of OOP as “**bundling data and behavior into meaningful units**” - each unit models something real.

What You Can Now Do

You can now design your own classes and instantiate multiple objects to model real-world entities, making your Python programs cleaner and more modular.

Next Steps

To deepen this knowledge explore the below mentioned resources **Additional resources:**

- [Python OOP Tutorial - Real Python](#)
- [Official Python Documentation: Classes](#)

Quick Reference Card (Optional)

Concept	Definition	Example
Class	Blueprint for creating objects	<code>class Dog:</code>
Object	Instance of a class	<code>my_dog = Dog()</code>
Constructor	Initializes new objects	<code>def __init__(...)</code>
Attribute	Object data (variables)	<code>self.name</code>
Method	Object behavior (functions)	<code>def bark(self):</code>